華南理工大學

South China University of Technology

本科毕业设计（论文）

基于联邦学习的车联网入侵检测系统设计与实现

学　　院	计算机学院（待核对）
专　　业	计算机科学与技术
学生姓名	待补充
学生学号	待补充
指导教师	待补充（指导教师）
提交日期	2026 年 4 月 17 日

华南理工大学

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

作者签名：

日期：2026 年 4 月 17 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保存并向国家有关部门或机构送交论文的复印件和电子版，允许学位论文被查阅；学校可以公布学位论文的全部或部分内容，可以允许采用影印、缩印或其它复制手段保存、汇编学位论文。本人电子文档的内容和纸质论文的内容相一致。

作者签名：

日期：2026 年 4 月 17 日

指导教师签名：　　日期：2026 年 4 月 17 日

作者联系电话：待补充　　电子邮箱: 待补充

摘　要

面向车联网环境的入侵检测任务具有节点分散、数据异构、通信受限和隐私敏感等特点，传统集中式训练方案难以同时兼顾检测性能、通信效率与数据安全。针对上述问题，本文围绕“基于联邦学习的车联网入侵检测系统设计与实现”开展研究，依托现有工程代码构建了包含数据预处理、联邦训练与本地评估三条命令行链路的原型系统。系统采用轻量化 CNN-LSTM 模型作为检测器，在联邦训练阶段实现了 FedAvg/FedProx 聚合框架，并引入基于算力、电量与信道质量的动态节点选择机制，同时结合 Top-K 稀疏化与定点量化策略以降低参数上传开销；此外，系统还支持配置加载、日志记录、模型检查点保存及证据报告落盘，便于实验复现与结果追踪。

基于仓库内置演示数据与实验报告的验证结果表明：相较于未压缩且全节点参与的基线 FedAvg 方案，改进方案在当前小样本实验中将总训练时间降低约 12.27%，平均轮次耗时降低约 8.51%，总通信量降低约 80.28%，而 Accuracy、Recall 与 F1-score 仍保持在 0.60、1.00 与 0.75 的同一水平。实验结果说明，所设计系统能够在不上传原始数据的前提下完成联邦建模，并在演示性场景中体现出较好的综合收益。考虑到当前实验样本规模较小，正式论文阶段仍需在真实 VeReMi 或 Car-Hacking 数据集上开展多次重复实验，以进一步验证系统稳定性与泛化能力。

关键词：　联邦学习；车联网；入侵检测；CNN-LSTM；参数压缩

Abstract

Intrusion detection in the Internet of Vehicles is challenged by geographically distributed nodes, heterogeneous local data, constrained communication resources, and strict privacy requirements. To address these issues, this thesis studies the design and implementation of a federated learning-based intrusion detection system for the Internet of Vehicles. Based on the existing project codebase, a prototype system is organized into three command-line workflows: data preprocessing, federated training, and local evaluation. A lightweight CNN-LSTM model is used as the detector, while the federated training module supports the FedAvg/FedProx aggregation framework. In addition, a dynamic client selection strategy based on computing capability, battery level, and channel quality is introduced, and Top-K sparsification together with fixed-point quantization is adopted to reduce model update transmission cost. The system also provides configuration loading, logging, checkpoint persistence, and evidence report generation to support reproducibility.

Experimental evidence from the built-in demo dataset and archived reports shows that, compared with an uncompressed full-participation FedAvg baseline, the improved method reduces total training time by about 12.27%, decreases mean round duration by about 8.51%, and lowers overall communication volume by about 80.28% in the current small-sample experiment, while maintaining the same Accuracy, Recall, and F1-score at 0.60, 1.00, and 0.75 respectively. These results indicate that the proposed system can complete collaborative model training without uploading raw vehicular data and can achieve a better trade-off between communication efficiency and detection effectiveness in a demonstrative scenario. Since the present experiment is still limited by sample size, further repeated evaluation on real VeReMi or Car-Hacking datasets is required in the formal thesis stage.

Keywords: Federated learning, Internet of Vehicles, Intrusion detection, CNN-LSTM, Communication compression

目　录

摘　要 ..... **I**

Abstract ..... **II**

目　录 ..... **III**

插图目录 ..... **V**

表格目录 ..... **VI**

第一章　绪论 **1**

1.1　研究背景与意义 1

1.2　国内外研究现状 2

1.3　研究内容与目标 2

1.4　论文结构安排 3

第二章　相关技术与理论基础 **4**

2.1　车联网与入侵检测概述 4

2.2　联邦学习基本原理 4

2.3　轻量化检测模型与训练优化机制 5

2.4　评价指标 6

2.5　本章小结 7

第三章　系统分析与总体设计 **8**

3.1　系统目标与使用场景 8

3.2　总体架构设计 9

3.3　数据流与运行流程设计 10

3.4　本章小结 12

第四章　系统实现 **13**

4.1　数据预处理实现 13

4.2　客户端切分与状态建模实现 14

4.3　联邦训练与聚合实现 15

4.4 参数压缩与通信统计实现 16
4.5 模型评估与报告落盘实现 17
4.6 本章小结 18

第五章 实验设计与结果分析 19
5.1 实验目标与研究问题 19
5.2 实验环境与参数设置 19
5.3 评价指标与对照方案 20
5.4 实验结果与分析 21
5.5 局限性与有效性讨论 22
5.6 本章小结 23

结论与展望 24
0.1 论文工作总结 24
0.2 工作展望 25

参考文献 26

致谢 27

附录 A 复现实验命令与结果文件说明 28
A.1 复现实验命令示例 28
A.1.1 数据预处理命令 28
A.1.2 改进方案训练命令 28
A.1.3 基线方案训练命令 28
A.1.4 模型评估命令 29
A.2 关键输出文件与证据路径 29
A.3 关键配置与参数补充说明 29

插图目录

3-1 车联网联邦入侵检测系统总体架构占位图 8
3-2 系统模块关系占位图 9
3-3 系统运行流程占位图 11

4-1 数据预处理流程占位图 14
4-2 联邦训练与聚合算法占位图 16
4-3 通信统计与压缩收益占位图 16

表格目录

3-1 系统核心模块与职责划分 10
3-2 关键配置参数占位表 11

4-1 客户端状态字段占位表 15
4-2 压缩机制关键参数占位表 17
4-3 系统关键输出文件占位表 18

5-1 实验主要参数设置 20
5-2 改进方案与基线方案的关键结果对比 21

A-1 附录中的关键输出文件说明 29

第一章 绪论

1.1 研究背景与意义

随着智能网联汽车、路侧单元与云平台之间的协同程度不断提高，车联网已经从单纯的车载通信场景扩展为融合感知、决策、控制与服务分发的复杂网络系统。车辆与车辆、车辆与路侧基础设施之间持续交换位置、速度、诊断状态与控制指令等数据，这一方面提高了交通效率与行车安全，另一方面也显著扩大了攻击面。一旦通信链路、车载终端或边缘节点遭受伪造报文、重放、拒绝服务或异常控制指令注入等攻击，轻则导致业务异常，重则可能引发交通安全风险。因此，在车联网环境中构建及时、可靠且具备工程可落地性的入侵检测机制，已经成为智能交通安全研究中的重要问题之一 [1-2]。

传统入侵检测系统通常依赖集中式日志汇聚与统一训练流程，即将多源数据收集到中心节点后进行特征工程、模型训练与全局判决。这类方法在静态或可控网络环境中较易实施，但在车联网场景下往往面临三方面限制。首先，车辆节点的算力、能耗与链路状态存在显著差异，持续上传原始数据会带来较高的带宽成本与调度压力。其次，车载数据天然包含位置轨迹、行为模式等敏感信息，直接集中存储存在隐私泄露风险。最后，不同车辆或不同道路环境下的数据分布并不一致，简单套用中心化训练策略往往难以兼顾泛化能力、通信代价与部署效率 [3-4]。

联邦学习为上述问题提供了新的解决思路。该方法强调“数据不出本地”，各参与节点仅上传模型更新或统计信息，由中心协调端完成聚合，从而在一定程度上兼顾数据可用性与隐私保护需求 [5-6]。然而，联邦学习并不天然适用于车联网入侵检测。一方面，车辆节点参与训练具有动态性与异质性，若所有节点在每轮中无差别参与，将导致训练时延和通信量迅速升高；另一方面，模型更新本身也会带来不小的传输负担，若缺少压缩和筛选机制，则系统整体效率仍然受限。基于此，本文围绕联邦学习与车联网入侵检测的结合问题，研究一个面向工程实现的原型系统，并在真实代码基础上讨论其设计、实现与实验验证过程。

从工程应用角度看，本研究具有以下意义：其一，为车联网入侵检测任务提供了一套从数据预处理、联邦训练到本地评估的完整链路，有助于将算法验证与系统实现结合起来；其二，通过引入节点选择、Top-K 稀疏化与定点量化等策略，探索在资源受限网络中降低通信开销的可行路径；其三，将配置快照、日志、运行元数据与实验报告统一沉淀到输出目录中，为后续复现实验、撰写论文和补充正式数据集验证提供了证据基础。由于当前开题报告与任务书正文尚未完成结构化提取，相关原始措辞与文献脉络在本文中暂以受控占位形式保留，后续可结合正式材料进一步细化 [7]。

1.2 国内外研究现状

围绕车联网安全问题，国内外研究已从早期基于规则与签名的检测方法，逐步发展到结合统计学习和深度学习的智能化检测方法。规则驱动方法实现简单、解释性较强，适用于已知攻击模式的快速识别，但对未知攻击与复杂异常行为的适应能力有限。随着车载总线日志、网络流量与多源传感数据逐渐丰富，研究者开始采用支持向量机、随机森林、卷积神经网络、循环神经网络等模型对异常流量与攻击事件进行识别，以期提升复杂场景下的检测精度与泛化能力 [8-9]。不过，这类方法多数默认数据可以集中收集，较少充分考虑车联网环境中持续在线训练、链路受限与隐私保护并存的现实约束。

在联邦学习方向，已有研究证明该范式能够在医疗、金融、移动终端与边缘计算等场景中支持跨节点协同建模，其中 FedAvg 是最具代表性的基础聚合方法之一，其思想是各客户端完成若干轮本地更新后，由服务器对模型参数进行加权平均 [5-6]。随后，FedProx 等方法通过在本地目标函数中加入近端正则项，缓解非独立同分布数据和系统异质性带来的训练不稳定问题 [10]。这些方法为车联网入侵检测提供了理论基础，但直接迁移到车辆场景时仍面临节点数量波动大、训练参与机会不均衡、更新上传成本高等挑战。

近年来，也有研究尝试将联邦学习用于工业互联网、物联网和车联网的异常检测任务，通过分布式训练方式提升数据利用率并降低原始数据外泄风险 [11-12]。然而，已有方案通常聚焦于联邦学习框架本身或单一模型效果，对工程侧的完整链路描述相对不足，例如缺少标准化的数据预处理流程、配置管理机制、训练证据留存方式以及通信统计分析。同时，一些工作虽然关注压缩通信或客户端选择，但往往分别讨论节点筛选与模型压缩，缺少面向统一系统的综合实现与对照验证。基于这一现状，本文将研究重点放在“可运行、可复现、可分析”的车联网联邦入侵检测原型构建上，并在实验部分讨论改进策略相对标准 FedAvg 基线的收益与局限。

1.3 研究内容与目标

本文以当前仓库中的 Python 项目和华南理工大学论文模板为基础，面向“基于联邦学习的车联网入侵检测系统设计与实现”这一题目组织论文内容。结合现有代码与实验材料，本文的研究内容主要包括以下几个方面。

第一，构建完整的系统运行链路。项目在 ‘main.py‘ 中提供统一命令行入口，支持 ‘preprocess‘、‘train‘ 与 ‘test‘ 三种模式；其中，预处理阶段负责读取本地表格数据并生成标准化训练集、验证集和标签映射，训练阶段负责联邦轮次调度、客户端本地更新与全局聚合，测试阶段则加载全局模型并对验证集进行评估。围绕这一主流程，系统还引入

了配置加载、随机种子设置、日志记录与运行元数据保存等支撑模块，使得实验过程具备较好的复现性与可追踪性。

第二，设计适用于车联网场景的联邦训练改进策略。现有实现以 FedAvg/FedProx 框架为基础，在客户端选择阶段综合考虑节点算力、电量与信道质量，并通过轻量公平性惩罚减弱同一节点被频繁选中的问题；在模型更新上传阶段，对参数增量执行 Top-K 稀疏化与定点量化，以降低通信量并形成可度量的压缩收益。上述策略并非脱离代码的理论设想，而是已经在 federated_learning.py 中落成具体实现，并可通过训练历史与报告文件进行核验。

第三，构建轻量化的入侵检测模型与评价体系。项目使用 ‘models.py‘ 中定义的 ‘LightweightCNNLSTM‘ 作为检测器，通过一维卷积提取局部特征，再由 LSTM 建模序列关系，最终输出攻击类别预测。为全面分析系统效果，本文不仅关注准确率、召回率、精确率和 F1 值等检测指标，还结合通信量、训练耗时、隐私代理分数等工程指标，对改进方案与基线方案进行综合比较。

基于以上内容，本文的研究目标可以概括为：在不上传原始车联网数据的约束下，设计并实现一套具备数据预处理、联邦训练、本地评估与证据报告能力的入侵检测系统；在现有演示实验条件下验证动态节点选择与参数压缩机制对通信成本和训练效率的改善作用；同时保持对系统边界的审慎表述，不将当前小样本结果直接推广为大规模真实场景的最终结论。对于开题报告、任务书中尚待补充的原始表述，本文先按已验证事实给出概括性描述，后续再根据正式材料补全文字与文献来源。

1.4 论文结构安排

本文正文共分为五章。第一章为绪论，介绍课题背景、研究意义、国内外研究现状以及本文的研究内容与目标。第二章为相关技术与理论基础，重点阐述车联网入侵检测、联邦学习、FedAvg/FedProx、轻量化 CNN-LSTM 模型、节点选择机制、参数压缩与评价指标等基础内容。第三章为系统分析与总体设计，结合项目实际代码结构说明系统目标、使用场景、模块划分、数据流和运行流程。第四章为系统实现，围绕数据预处理、客户端切分、联邦训练、模型聚合、通信统计与结果落盘等关键实现细节展开说明。第五章为实验设计与结果分析，给出实验目标、参数设置、基线方案、结果对比和局限性讨论。正文之后为结论与展望、参考文献、致谢及附录，用于总结全文、保留佐证材料并支持后续继续完善。

第二章 相关技术与理论基础

2.1 车联网与入侵检测概述

车联网是由车载终端、路侧单元、边缘节点、云平台以及多类通信协议共同组成的开放式协同网络，其核心特征在于高移动性、强时变性和多主体异构协作。与传统企业网络相比，车联网中的节点地理分布更广，通信链路质量波动更明显，设备侧资源约束也更强。一旦攻击者利用伪造身份、恶意报文、重放数据或异常控制指令破坏通信流程，不仅会影响信息服务质量，还可能干扰车辆控制决策，因此安全防护必须兼顾实时性、鲁棒性和低开销 [1-2]。

入侵检测系统的任务是在海量运行数据中识别正常行为与异常行为，并尽可能降低漏报与误报。现有车联网入侵检测方案通常包括基于规则、基于统计分析以及基于机器学习三类思路。规则法适合识别已知模式，统计法关注行为偏离程度，而机器学习方法更擅长从复杂特征中学习潜在模式。对于当前项目而言，检测对象主要体现为表格化的车联网日志或网络流量记录，系统通过对原始字段执行缺失值处理、标签规范化与特征提取，再利用深度模型完成分类判别。这种“数据驱动检测”范式更适合在复杂环境下扩展，但其前提是能够在保证隐私和通信代价可控的前提下完成模型训练。

在车联网环境中，入侵检测还呈现出几个典型难点。第一，数据分布具有明显异构性，不同车辆、不同道路环境下采集到的样本差异较大；第二，车辆节点参与训练的时间窗口有限，且算力、电池状态与信道质量并不稳定；第三，原始数据中可能包含敏感位置信息、控制状态或行为轨迹，不适合直接集中上传。由此可见，车联网入侵检测不仅是模型分类问题，更是一个涉及分布式计算、资源调度与工程部署约束的系统问题。

2.2 联邦学习基本原理

联邦学习是一种在多参与方之间协同训练模型的方法，其基本思想是将训练数据保留在本地，仅上传模型参数或梯度更新，由中心协调端完成聚合并向各节点分发新模型 [5-6]。对于车联网场景，这种方法能够减少原始数据跨节点流动，有助于满足“数据不出车端”的基本要求。结合当前项目实现，联邦训练流程大致可概括为：首先完成数据预处理并生成全局训练集与验证集；其次将训练集按客户端数量切分到本地文件；然后由协调端在每一轮中根据节点评分选择部分可用客户端参与训练；各客户端在本地执行若干轮梯度更新后，将更新后的参数上传；最后由服务器端执行聚合，并在验证集上评估全局模型效果。

FedAvg 是当前项目的基础聚合思路，其核心在于对各客户端本地训练后的模型参数进行按样本数加权平均。若第 t 轮选中的客户端集合记为 S_t，第 i 个客户端样本量为 n_i，本地训练得到的模型参数为 $w_i^{(t+1)}$，则全局模型更新可写为

$$w^{(t+1)} = \sum_{i \in S_t} \frac{n_i}{\sum_{j \in S_t} n_j} w_i^{(t+1)} \tag{2-1}$$

在 federated_learning.py 中，aggregate_weights() 正是按照上述思想对浮点参数执行加权求和，因此本文后续关于聚合机制的描述均以这一实现为依据。

考虑到车联网数据往往并非独立同分布，当前实现还兼容 FedProx 形式的近端正则化策略。当 fedprox_mu 大于零时，本地训练损失函数会在普通分类损失之外，增加当前模型参数与全局参考参数之间的距离惩罚，以减弱局部训练过度偏离全局目标的问题。其思想可以表示为

$$\mathcal{L}_{\text{local}} = \mathcal{L}_{\text{task}} + \frac{\mu}{2} \sum_k \left\| w_k - w_k^{(t)} \right\|_2^2 \tag{2-2}$$

代码中的 _compute_fedprox_penalty() 即实现了这一项。虽然当前实验记录主要展示的是改进方案与标准 FedAvg 基线，但 FedProx 的接口已经在系统中预留，为后续更丰富的对照实验提供了支持。

2.3 轻量化检测模型与训练优化机制

当前项目采用 ‘LightweightCNNLSTM‘ 作为入侵检测模型。该模型首先使用两层一维卷积与批归一化模块提取局部特征，再通过最大池化降低特征长度，随后把卷积输出送入 LSTM 以建模序列关联，最后利用两层全连接分类器输出预测结果。与参数规模更大的深层模型相比，该结构兼顾了特征提取能力与部署轻量化需求，适合在资源受限环境下作为车联网入侵检测器。根据现有报告文件，模型总参数量为 8402 个，模型大小约为 33608 字节，能够为后续通信统计和压缩分析提供量化基础。

为了适应车联网环境中节点状态差异明显的特点，系统在每一轮训练前会对可用客户端执行三维度动态节点选择。score_available_clients() 使用节点算力、电量水平和信道质量计算综合评分，并在此基础上引入轻量公平性惩罚，避免同一节点连续多轮被优先选中。若以 c_i、b_i、h_i 分别表示第 i 个客户端的算力、电量与信道质量，w_c、w_b、w_h 表示对应权重，t_i 表示该节点历史入选次数，则其最终评分可概括为

$$s_i = \max\left(0, \frac{w_c c_i + w_b b_i + w_h h_i}{w_c + w_b + w_h} - \min(0.05 t_i, 0.2)\right) \tag{2-3}$$

系统随后按评分从高到低排序，并根据 client_fraction 确定当轮参与节点数。该机制体现了车联网训练中“资源感知 + 公平性约束”的基本思想。

除节点筛选外，参数压缩也是降低联邦训练通信压力的重要手段。当前实现的 _compress_client_update() 首先计算客户端模型与全局模型之间的参数增量，然后对增量向量选取绝对值最大的 Top-K 元素，仅保留最具代表性的更新分量；在此基础上，系统还支持 8/16/32 位的定点量化，其中 32 位相当于不量化，8 位和 16 位则通过缩放、离散化与反量化重建近似更新值。该策略能够在保持主要更新方向的同时显著减少实际传输字节数。对于论文分析而言，Top-K 稀疏化体现了“只发送重要参数”的思想，而定点量化体现了“用更少比特表达更新值”的思想，两者叠加构成了当前项目中的通信优化核心。

需要指出的是，当前代码中所谓“隐私代理分数”并非严格意义上的差分隐私或密码学安全证明，而是一种结合数据本地保留程度与模型更新压缩收益的代理指标。系统默认原始数据留在本地，因此基础分与数据本地性相关；同时，若压缩带来更少的上传信息量，则代理分数相应提高。该定义更适合用于工程比较，而不宜直接解释为形式化隐私保障。

2.4 评价指标

为了从检测效果与系统开销两个层面对方案进行评价，本文采用分类性能指标与工程性能指标相结合的方式。对二分类任务而言，准确率（Accuracy）表示总体预测正确的样本比例；精确率（Precision）衡量被判为攻击的样本中真实攻击样本所占比例；召回率（Recall）反映真实攻击样本被成功识别的比例；F1 值则是精确率与召回率的调和平均。其定义分别可写为

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{2-4}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2-5}$$

其中，TP、TN、FP、FN 分别表示真正例、真反例、假正例和假反例。metrics.py 中的 calculate_classification_metrics() 即按这一思路计算 Accuracy、Precision、Recall 和 F1 值；当标签为二分类时，系统还会进一步根据混淆矩阵计算假正率（False Positive Rate）。

除检测性能外，联邦学习系统还必须关注通信与计算开销。当前项目通过训练历史与联邦报告记录以下指标：总训练时间、平均轮次耗时、平均本地训练耗时、原始通信量、压缩后通信量以及总体通信压缩率。总训练时间与平均轮次耗时可反映整体吞吐能

力，原始通信量与压缩后通信量则直接体现参数压缩策略的收益。对于论文实验部分，这些指标将与 Accuracy、Recall 和 F1 值共同构成改进方案和基线方案的对照分析依据。

另外，系统在每轮训练后还会生成隐私代理分数，用于反映“数据保留在本地”与“上传更新量下降”这两类因素的综合效果。虽然该指标不具备严格理论证明，但在当前工程原型中能够辅助说明改进方案在减少上传信息暴露面的同时，是否仍能保持可接受的检测性能。因此，后续实验分析将采用“检测性能、通信收益、训练收益、隐私代理”四个维度进行综合讨论。

2.5 本章小结

本章围绕本文所需的关键理论基础进行了说明。首先，分析了车联网环境中的安全挑战与入侵检测任务特点，指出该问题不仅涉及分类模型设计，还受到分布式部署、资源约束和隐私要求的共同影响；其次，介绍了联邦学习的基本流程，以及 FedAvg 和 FedProx 在当前系统中的实现意义；随后，结合代码中真实存在的 ‘LightweightCNNLSTM‘、动态节点选择、Top-K 稀疏化和定点量化机制，阐述了系统实现所依托的主要技术路线；最后，对分类指标、通信指标和隐私代理指标进行了统一说明。以上内容为第三章的系统分析与总体设计奠定了理论与术语基础。

第三章 系统分析与总体设计

3.1 系统目标与使用场景

本文设计的系统面向车联网环境中的分布式入侵检测任务，其核心目标不是追求单一指标的极端最优，而是在可运行、可复现和可验证的前提下，构建一条覆盖数据预处理、联邦训练和本地评估的完整工程链路。结合现有代码，系统需要同时满足以下目标：第一，能够从本地表格化车联网数据中自动发现可处理文件，并完成标签规范化、特征提取与训练/验证数据集导出；第二，能够在多客户端场景下执行联邦训练，支持动态节点选择、样本数加权聚合以及参数压缩；第三，能够对全局模型进行评估，并将配置快照、日志、训练历史、评估结果和联邦训练报告统一落盘，便于论文写作与实验复现。

从应用场景来看，该系统适用于具备边缘节点或协调服务器的车联网安全监测环境。在该场景中，不同车辆或车载终端会持续产生与通信行为、控制指令或状态变化相关的数据。由于原始数据可能包含敏感内容，系统不直接上传完整样本，而是先在本地完成预处理与模型更新，再由中心端聚合模型参数。当前实现更接近“联邦训练流程仿真与原型验证”而非真实车辆网络部署：客户端训练过程在单机环境中顺序模拟完成，重点在于验证模块边界、输出产物和指标采集机制是否完整，而不是构建大规模实网分布式调度平台。

结合代码可知，系统的使用边界也较为明确。其一，当前输入数据为 CSV、TSV 或 Excel 等表格文件，系统假设原始数据已经以离线文件形式可获取；其二，训练与评估依赖预处理产物，因此在执行 ‘train‘ 和 ‘test‘ 模式之前必须先完成 ‘preprocess‘ 模式；其三，系统采用验证集进行效果评估，尚未单独提供独立测试集流程；其四，隐私保护主要体现为“数据不出本地”和“更新量压缩”，并未实现密码学意义上的安全聚合或严格差分隐私机制。上述边界决定了本文后续分析必须以工程原型视角展开，而不能将现有实现表述为完备的生产级车联网安全平台。

为便于后续补充正式绘图，本章预留系统总体架构图位置，如图3-1所示。

系统总体架构图占位。
建议后续绘制“数据源层—预处理层—联邦训练协调层—车端本地检测层—结果与证据输出层”的五层结构图，突出 ‘main.py‘、‘data_processor.py‘、‘federated_learning.py‘、‘models.py‘、‘metrics.py‘ 与 ‘outputs/‘ 之间的关系。

图 3-1 车联网联邦入侵检测系统总体架构占位图

3.2 总体架构设计

从代码组织方式看，系统可划分为入口编排、配置管理、数据预处理、联邦训练、模型定义、指标计算、运行支撑和结果输出八个核心部分。其中，‘main.py‘ 负责统一命令行入口与流程调度；‘config.py‘ 负责 TOML/JSON 配置加载、相对路径解析以及运行时配置对象构建；‘data_processor.py‘ 面向原始表格数据完成清洗与特征工程；‘federated_learning.py‘ 负责客户端选择、本地训练、参数聚合、压缩统计与训练历史维护；‘models.py‘ 提供轻量化 CNN-LSTM 模型定义与评估接口；‘metrics.py‘ 则封装分类指标计算逻辑；‘runtime_utils.py‘ 与 ‘app_logging.py‘ 提供随机种子设置、元数据保存和日志输出支持；所有运行证据最终沉淀至 ‘outputs/‘ 目录。

若从职责分层角度进一步抽象，可以将系统分为三层。第一层为数据与配置层，对应原始数据文件、配置文件以及运行时参数；第二层为联邦训练与检测核心层，对应预处理、模型训练、聚合和评估逻辑；第三层为结果与证据层，对应训练历史、模型检查点、日志、评估结果和联邦训练报告等产物。三层之间通过数据文件与配置对象连接：配置层将运行参数封装为 ‘AppConfig‘，核心层据此执行处理逻辑，结果层再将运行证据持久化。该结构既有助于保持模块边界清晰，也方便在后续扩展真实数据集、更多模型或更复杂实验设置时复用既有流程。

在总体架构中，各模块的接口关系较为稳定。‘PathConfig‘ 为所有产物提供统一路径约定，如预处理文件、日志目录、报告目录和模型检查点；‘DatasetConfig‘ 定义数据集名称、标签列和验证集比例；‘TrainingConfig‘ 定义轮数、客户端数量、局部训练轮数、压缩比例、量化位宽和选择权重等关键参数；‘ClientState‘、‘PreprocessResult‘ 与 ‘EvalResult‘ 则在模块间传递客户端状态、预处理结果与评估结果。由此形成了“配置对象驱动流程、数据结构串联模块、输出目录保留证据”的系统设计风格。

表3-1 总结了系统主要模块及其功能定位。

为便于后续在论文中补充更加规范的模块关系图，本节同时预留模块关系图位置，如图3-2所示。

模块关系图占位。
建议后续绘制入口层、配置层、预处理层、训练层、评估层、日志/报告层之间的调用关系，并突出 ‘AppConfig‘、‘ClientState‘、‘PreprocessResult‘ 和 ‘EvalResult‘ 的数据流作用。

图 3-2 系统模块关系占位图

表 3-1 系统核心模块与职责划分

模块	主要职责
‘main.py‘	统一 CLI 入口，解析参数，构建配置对象，按模式调度预处理、训练与测试流程。
‘config.py‘	加载 TOML/JSON 配置，解析路径，定义 ‘AppConfig‘ 及其子配置对象。
‘data_processor.py‘	发现数据文件，规范化标签与列名，提取数值、时间和字符串特征，导出训练/验证数据。
‘federated_learning.py‘	切分客户端数据，执行节点选择、本地训练、参数聚合、通信统计与联邦报告生成。
‘models.py‘ 与 ‘metrics.py‘	定义轻量化 CNN-LSTM 模型并计算 Accuracy、Precision、Recall、F1 等指标。
‘runtime_utils.py‘ 与 ‘app_logging.py‘	负责随机种子控制、元数据保存、控制台与文件日志输出。

3.3 数据流与运行流程设计

系统运行流程由 ‘main.py‘ 的 ‘main()‘ 统一控制。程序启动后，首先根据用户是否提供 ‘–config‘ 参数执行默认配置加载；若配置文件存在，则 ‘load_parser_defaults()‘ 先从 TOML 或 JSON 文件中提取默认值，再由命令行显式参数覆盖同名字段，最终通过 ‘build_configs()‘ 生成 ‘AppConfig‘ 对象。随后，系统调用 ‘ensure_runtime_dirs()‘ 创建输出目录体系，初始化日志，设置随机种子，并保存配置快照和运行元数据。这一处理保证了无论执行哪一种模式，运行前都能保留完整的上下文信息。

预处理模式对应的主要数据流为“原始文件发现—数据合并—列规范化—缺失值处理—标签标准化—特征提取—训练/验证切分—产物落盘”。在该阶段，系统会扫描数据目录中的 CSV、TSV 和 Excel 文件，将其拼接为统一数据集，并根据标签列定义自动识别 ‘label‘、‘class‘、‘attack‘ 等别名字段。随后，数值型、时间型和字符串型字段分别采用不同方式完成特征化，最终生成 ‘train_processed.csv‘、‘val_processed.csv‘ 和 ‘label_mapping.json‘ 三类核心文件，为后续训练与测试阶段提供标准输入。

训练模式对应的主要数据流为“预处理产物读取—客户端切分—节点评分与选择—本地训练—更新压缩—全局聚合—轮次评估—训练历史记录—报告输出”。具体而言，系统会先通过 ‘load_preprocess_result()‘ 读取预处理结果，再将训练集切分为多个客户端文件，并为每个客户端维护资源状态。随后，协调端按轮次执行评分与选点，对选中客户端进行顺序本地训练，获得参数更新后执行 Top-K 稀疏化与量化，再由协调端按照样本数加权平均生成新的全局模型。每轮结束后，系统会立即记录本地损失、训练时延、通信统计、隐私代理分数和验证结果，并将累计训练历史写入 ‘training_history.json‘。

测试模式对应的主要数据流相对直接，即“读取预处理产物—加载全局模型检查点—在验证集上评估—保存评估结果”。当前系统并没有另外构造独立测试集，而是调用 ‘models.py‘ 中的评估接口对验证集执行推理与指标计算，并将结果写入 ‘reports/evaluation.json‘。这一设计更适合小规模工程验证：它可以快速检验模型是否能够完成完整的训练和推理闭环，同时保留混淆矩阵、样本量与延迟等指标，为后续正式实验提供依据。

从输出目录组织来看，系统将运行证据明确分为数据产物、模型产物、日志产物和报告产物四类。数据产物包括训练集、验证集与标签映射；模型产物主要是全局模型检查点；日志产物集中保存在 ‘logs/app.log‘；报告产物包括配置快照、运行元数据、评估结果以及联邦训练 JSON/Markdown 双格式报告。通过这种目录约定，系统实现了从输入到输出的证据闭环。

为了便于后续继续增强排版，本节预留运行流程图和关键配置参数表位置，分别见图3-3与表3-2。

训练流程图占位。 建议后续绘制“配置加载 → 预处理 → 客户端切分 → 节点选择 → 本地训练 → 压缩上传 → 聚合评估 → 结果落盘”的流程图。

图 3-3 系统运行流程占位图

表 3-2 关键配置参数占位表

参数类别	代表字段	说明
数据配置	‘dataset_name‘、‘label_column‘、‘validation_split‘	定义数据集名称、标签列名及训练/验证集划分比例。
训练配置	‘global_rounds‘、‘num_clients‘、‘client_fraction‘、‘fedprox_mu‘	定义联邦训练轮数、客户端数量、参与率和近端正则系数。
压缩配置	‘compression_topk_ratio‘、‘quantization_bits‘	定义上传更新的稀疏化比例与量化位宽。
节点选择配置	‘selection_weight_compute‘、‘selection_weight_battery‘、‘selection_weight_channel‘	定义三维节点选择机制中的资源权重。
运行配置	‘seed‘、‘deterministic‘、‘run_name‘、‘log_level‘	定义随机性控制、运行命名和日志行为。

3.4 本章小结

本章从系统目标、使用场景、总体架构、模块边界、数据流和运行流程等方面对车联网联邦入侵检测系统进行了总体设计。分析表明，当前项目采用了较为清晰的模块化结构：由统一入口驱动配置加载和流程编排，以预处理、联邦训练和本地评估三类模式构成主链路，并通过结构化配置对象和输出目录约定实现模块协作与证据沉淀。该设计既满足了本科毕设中“系统分析与总体设计”章节的完整性要求，也为后续的实现细节展开提供了清晰框架。下一章将在此基础上，结合 ‘data_processor.py‘、‘federated_learning.py‘、‘models.py‘、‘runtime_utils.py‘ 和 ‘app_logging.py‘ 等模块，进一步说明系统的关键实现机制。

第四章 系统实现

4.1 数据预处理实现

数据预处理由 data_processor.py 中的 DataProcessor 类负责实现，是整套系统中最先执行且最基础的环节。该模块首先通过 discover_data_files() 在指定目录中递归发现可处理数据文件，支持 CSV、TSV 与 Excel 三类表格输入；若路径不存在、格式不受支持或目录中没有可处理文件，系统会直接抛出异常，从而避免在后续训练阶段出现隐蔽错误。随后，_load_dataset() 将多个文件读取并合并为统一数据表，为进一步清洗与特征构造提供基础。

在字段清洗方面，预处理模块采用了“列名规范化—标签列识别—缺失值填补”的处理流程。_normalize_columns() 会将原始列名统一转换为小写并用下划线连接，减少不同数据源列命名差异带来的影响；_ensure_label_column() 则会自动识别 label、labels、class、attack、attack_type 等常见别名，并统一映射为系统内部使用的标签列。之后，_handle_missing_values() 按字段类型进行差异化填补：数值列优先转换为数值并以中位数补全，字符串列采用众数或 unknown 填充，从而保持数据矩阵的完整性。

在标签处理与特征提取方面，系统采用了与车联网日志数据相适配的规则式方案。_standardize_labels() 会把正常类标签统一到 benign，攻击类标签统一到 attack，其余标签则经过正则清洗后保留；随后，_extract_features() 根据字段内容自动选择特征化方式：数值列直接转为浮点特征，时间类字段转换为时间戳，普通字符串字段则提取长度、编码结果、词元计数和唯一字符数等统计特征；若字段名包含 payload、data、frame 或 message 等关键词，还会进一步计算十六进制长度和非零字节比例。相比完全手工设计特征的方式，该实现更强调对多源表格数据的适配性与自动化程度。

完成特征与标签编码后，系统通过 _build_label_mapping() 构建标签映射，并调用 _split_dataset() 按配置比例划分训练集和验证集。若各类样本数量满足条件，切分过程会优先使用分层抽样，以尽量保持类别分布稳定；否则回退到普通随机划分。最终，_persist_artifacts() 将结果分别写入 train_processed.csv、val_processed.csv 和 label_mapping.json。至此，预处理阶段完成了从原始多文件数据到标准训练输入的转换。

为便于后续补充正式流程图，本节预留预处理流程伪代码占位，如图4-1所示。

数据预处理流程占位。
建议后续绘制“数据发现 → 列规范化 → 标签识别 → 缺失值处理 → 特征提取 → 数据切分 → 结果落盘”的伪代码或流程图。

图 4-1 数据预处理流程占位图

4.2 客户端切分与状态建模实现

在联邦训练开始之前，系统需要将统一训练集转换为多个客户端的本地训练上下文。该过程由 FedAvgProxOptimizer 中的 _prepare_client_states() 完成。若外部未显式传入带有训练路径的客户端状态列表，系统会根据 num_clients 参数把训练数据按索引均匀切分为若干子集，并将每个子集分别保存到 outputs/clients/client_i_train.csv 文件中。这样做有两个好处：一方面，后续本地训练过程可以直接通过文件路径读取客户端样本；另一方面，客户端切分结果会保留在磁盘中，便于实验排查与附录引用。

系统使用 schemas.py 中定义的 ClientState 数据结构来描述客户端状态。除了客户端编号、训练数据路径、样本数等基本信息外，该结构还维护节点可用性、算力水平、电量水平、信道质量、隐私本地性分数、上一次训练损失、训练耗时和最近一次选择评分等字段，为动态节点选择与训练过程分析提供数据支撑。需要强调的是，当前资源属性并非来自真实车端硬件监测，而是根据客户端编号按规则赋予的仿真值，其目的在于构造可重复的资源异构场景，而非精确刻画某一具体车辆设备的真实状态。

若已有外部构造的客户端状态传入系统，_prepare_client_states() 会优先复用这些对象，并通过 _hydrate_client_resources() 补齐缺失的算力、电量、信道质量和隐私本地性字段。这种实现体现了系统在接口层面的扩展性：既可以直接使用内置仿真资源生成逻辑，也可以在后续研究中替换为更真实的节点状态来源。客户端切分与状态建模的结果，将直接影响每轮联邦训练中的节点筛选与聚合权重分配，因此是连接数据预处理与联邦优化的关键桥梁。

表4-1 为客户端状态字段预留说明位置，便于后续补充更细致的参数描述。

表 4-1 客户端状态字段占位表

字段	说明
client_id、train_data_path、num_samples	描述客户端身份、本地训练数据位置和样本规模。
compute_capacity、battery_level、channel_quality	描述客户端在节点选择中的三维资源状态。
privacy_locality_score	表征数据保留在本地的隐私代理基础分。
last_loss、last_training_time_ms、last_selection_score	记录最近一次训练和选择过程中的关键统计量。
metadata	存储附加信息，如历史入选次数等。

4.3 联邦训练与聚合实现

联邦训练主流程由 FedAvgProxOptimizer.train() 驱动。方法执行时，系统首先读取预处理结果，构建全局模型与初始参数副本，再按设定轮数循环执行联邦通信轮次。在每一轮中，系统先调用 score_available_clients() 对所有可用节点进行评分，并根据资源状况与公平性惩罚计算综合分数；随后通过 select_clients() 按 client_fraction 从高到低选择参与节点。该选择机制并非随机采样，而是显式利用算力、电量和信道状态构造优先级，因此更符合车联网资源受限环境下的训练调度逻辑。

对于被选中的每个客户端，系统通过 _train_single_client() 执行本地训练。该函数会读取本地数据文件，构造 TensorDataset 和 DataLoader，再初始化 LightweightCNNLSTM 模型，并将全局参数加载为本地初始状态。本地优化器使用 Adam，任务损失为交叉熵。若配置中启用了 fedprox_mu，系统会通过 _compute_fedprox_penalty() 在分类损失之外追加参数偏移惩罚，从而抑制局部模型与全局参考模型的过度偏离。每个客户端本地训练完成后，系统会记录平均损失、训练时长和对应的选择证据，为后续分析节点选择与训练表现的关系提供依据。

当一轮中的全部客户端完成本地更新后，系统先对每个客户端的模型增量执行压缩重建，再由 aggregate_weights() 进行全局聚合。该函数按照客户端样本量对浮点参数执行加权平均，对非浮点张量则直接复制模板值。由此可见，当前系统使用的是基于样本权重的参数聚合，而非密码学意义上的安全聚合方案。全局聚合完成后，系统会立即在验证集上调用模型评估接口，并将本轮选中节点、平均本地损失、平均训练耗时、通信统计、隐私代理分数和验证结果一并写入训练历史中。

为了方便论文后续增加算法描述，本节预留联邦训练关键算法流程占位，如图4-2所示。

联邦训练与聚合算法占位。
建议后续以伪代码形式描述：初始化全局模型、客户端评分与选择、本地训练、更新压缩、按样本数聚合、验证评估、历史记录更新。

图 4-2 联邦训练与聚合算法占位图

4.4 参数压缩与通信统计实现

在车联网联邦训练中，模型参数更新的传输成本直接影响系统吞吐与部署可行性。为此，当前系统在 _compress_client_update() 中实现了“参数增量计算—Top-K 稀疏化—定点量化—重建统计”的压缩流程。具体而言，系统首先计算本地模型与全局模型之间的参数差值，将其展平为向量；随后根据 compression_topk_ratio 选取绝对值最大的若干分量，仅保留最关键的更新位置；若量化位宽设置为 8 或 16，则进一步对选中分量执行缩放、离散化与反量化重建，以减少表示更新所需的字节数；若位宽为 32，则可视为不量化。

压缩完成后，系统不仅恢复一个近似的客户端更新状态用于后续聚合，还会同步生成通信统计记录，包括原始字节数、压缩后字节数、保留比例和总压缩率等信息。随后，_summarize_round_communication() 会基于所有客户端压缩记录，计算该轮的原始通信量、压缩后通信量、相对本轮原始上传量的压缩率，以及相对“全体可用客户端均上传完整模型”这一基线的节省比例。这样的实现使得通信收益既可以从局部压缩效果观察，也可以从整体系统层面衡量。

在此基础上，系统还通过 _calculate_privacy_proxy_score() 构造隐私代理指标。该指标将客户端的本地性分数与压缩收益加权组合，反映“原始数据不出本地”与“上传更新量减少”对隐私暴露面的共同影响。需要强调的是，这一分数是工程分析用的代理量，而不是理论可证的隐私保障指标。因此，在论文中应将其用于相对比较，而不能将其描述为严格隐私证明。

为了便于后续补充具体图表，本节预留通信统计图和压缩参数表的位置，分别见图4-3与表4-2。

通信统计图占位。
建议后续展示不同方案下总通信量、平均轮次耗时和隐私代理分数的对比柱状图或折线图。

图 4-3 通信统计与压缩收益占位图

表 4-2 压缩机制关键参数占位表

参数或指标	说明
compression_topk_ratio	控制每次上传保留的参数增量比例。
quantization_bits	控制压缩值的量化位宽，当前支持 8、16、32 位。
original_bytes 与 compressed_bytes	分别表示未压缩更新和压缩后更新的字节数。
reduction_ratio	描述压缩后相对于原始更新的节省比例。
saving_against_full_participation	描述相对于全量节点完整上传基线的整体节省比例。

4.5 模型评估与报告落盘实现

模型评估主要通过 models.py 中的 evaluate() 与 metrics.py 中的 calculate_classification_metrics() 协同完成。评估阶段首先加载全局模型检查点，然后在验证集上执行批量预测，获得预测标签与损失值，最后计算 Accuracy、Precision、Recall、F1、假正率、样本量和混淆矩阵等指标。由于这些指标统一封装在 EvalResult 结构中，因此既可以作为训练过程中每一轮的验证结果，也可以作为独立测试模式下的最终评估结果。该设计使得训练阶段和测试阶段共享同一套评价口径，减少了实验对比中的统计偏差。

除评估外，系统还特别强调运行证据的自动落盘。runtime_utils.py 中的 save_json() 会把配置快照、运行元数据与评估结果保存为 JSON 文件，collect_run_metadata() 则记录模式、命令行参数、时间戳、Python 版本、平台信息和当前工作目录。与此同时，app_logging.py 中的 setup_logging() 统一配置控制台与文件日志，可根据配置选择普通文本格式或 JSON 日志格式。这样一来，系统不仅能输出最终结果，还能保留完整的运行上下文与日志轨迹，有利于问题定位和论文附录整理。

在联邦训练结束后，系统调用 _build_federated_report() 汇总训练历史、模型规模、通信收益、平均本地训练耗时、平均隐私代理分数和最终评估结果，并通过 _persist_federated_report() 同时生成 JSON 与 Markdown 两种格式的联邦训练报告。前者适合后续程序分析，后者适合直接作为实验记录证据或附录材料。配合 training_history.json、global_model.pt、config_snapshot.json 和 run_metadata.json 等文件，系统最终形成了较为完整的证据留存链条。

表4-3 为关键输出文件预留说明位置，便于后续在附录中进一步展开。

表 4-3 系统关键输出文件占位表

输出文件	作用
train_processed.csv、val_processed.csv	保存预处理后的训练集与验证集。
label_mapping.json	保存标签到数值编码的映射关系。
training_history.json	保存各轮次选点、训练损失、通信统计和验证结果。
global_model.pt	保存联邦训练后的全局模型检查点。
reports/evaluation.json	保存独立测试模式下的评估指标结果。
reports/federated_training_report.json、reports/federated_training_report.md	保存联邦训练汇总报告，支持程序读取和人工查看。
logs/app.log、config_snapshot.json、run_metadata.json	保存日志、配置快照和运行上下文信息。

4.6 本章小结

本章围绕系统实现层面展开说明，依次介绍了数据预处理、客户端切分与状态建模、联邦训练与聚合、参数压缩与通信统计，以及模型评估与报告落盘等关键机制。可以看出，当前项目虽然规模不大，但已经具备较完整的工程闭环：既能完成从原始数据到全局模型的训练流程，也能保留足够丰富的运行证据，为实验对比和论文写作提供支撑。基于这些实现细节，下一章将进一步结合现有实验设计文档、改进方案报告与基线方案报告，对系统的性能表现、综合收益和现阶段局限性进行分析。

第五章 实验设计与结果分析

5.1 实验目标与研究问题

实验部分的目标不是仅给出若干静态结果，而是验证第四章所述系统实现能否形成一条完整、可复现且具备分析价值的联邦入侵检测链路。结合 ‘docs/experimental_design.md‘ 与现有实验产物，本文主要围绕以下问题展开验证：其一，轻量化 CNN-LSTM 模型是否能够在当前车联网演示数据上完成基本的攻击识别任务；其二，引入动态节点选择、Top-K 稀疏化与定点量化后，系统通信开销能否显著下降；其三，在通信压缩和节点筛选存在的情况下，检测性能是否保持在可接受范围内；其四，在原始数据不出本地的前提下，系统是否能够通过隐私代理指标体现出相对基线更高的综合收益。

根据实验设计文档，本文将上述目标进一步对应为四个研究问题。首先，研究问题 RQ1 关注动态节点选择是否有助于降低联邦轮次训练时延；其次，RQ2 关注参数压缩机制是否能够有效减少总通信量；再次，RQ3 关注改进方案对 Accuracy、Recall 和 F1 值等检测性能的影响是否可接受；最后，RQ4 关注在数据本地保留与上传更新压缩同时存在时，隐私代理分数是否能够得到提升。由此可见，实验章节强调的是“检测性能—通信收益—训练收益—隐私代理”四维协同分析，而不是单一精度指标的横向比较。

需要说明的是，当前仓库中能够直接核验的对照结果主要包括改进方案与标准 FedAvg 基线方案。实验设计文档中虽然列出了集中式训练和 FedProx 等候选对照组，但对应结果尚未在现有仓库产物中完整沉淀。因此，本章分析将以可直接核验的两组报告为主，并把其余对照方案保留为后续正式论文阶段的扩展实验方向。

5.2 实验环境与参数设置

当前实验使用的数据基础来自项目内置的 ‘data/demo_vehicular.csv‘，并以 ‘VeReMi & Car-Hacking‘ 作为数据集名称写入配置和报告文件。需要指出的是，这里的数据集名称更多体现研究目标与后续适配方向，而当前仓库内直接提供的是演示性复现实验数据，用于验证工程链路和指标采集能力。预处理、训练和测试过程均通过 ‘main.py‘ 的命令行入口执行，推荐命令示例已整理于附录A.1。

从运行环境看，项目 README 中建议使用 Python 3.10 及以上版本，并通过虚拟环境安装依赖。默认配置文件 ‘configs/default.toml‘ 指定设备为 CPU，数据路径为 ‘data/‘，输出目录为 ‘outputs/‘，同时给出了客户端数量、全局轮数、Top-K 比例、量化位宽和节点选择权重等默认参数。由于当前仓库中的实验结果主要用于工程验证，本章不把具体

硬件平台作为核心比较变量，而是将平台信息交由 ‘run_metadata.json‘ 自动记录，从而避免不同临时运行环境对结论表述造成混淆。

为了和已有报告保持一致，本文重点分析两组可核验方案：一组为改进方案，即动态节点选择 + Top-K 稀疏化 + 8 位量化；另一组为基线方案，即全客户端参与且不压缩上传更新的标准 FedAvg 形式。两组方案的关键参数设置如表5-1所示。

表 5-1 实验主要参数设置

参数项	改进方案	基线方案
数据集标识	VeReMi & Car-Hacking	VeReMi & Car-Hacking
客户端数量	4	4
联邦轮数	3	3
客户端参与率	0.75	1.0
Top-K 比例	0.2	1.0
量化位宽	8 bit	32 bit
节点选择权重	0.4 / 0.3 / 0.3	0.4 / 0.3 / 0.3
聚合方式	FedAvg/FedProx 框架下的加权聚合	标准 FedAvg 对照
结果目录	‘outputs/‘	‘outputs/baseline_fedavg/‘

需要特别说明的一点是，基线方案虽然仍保留了节点评分与选择证据结构，但由于 ‘client_fraction=1.0‘，所有可用客户端都会参与训练，因此评分结果不会影响是否被选中；相较之下，改进方案的 ‘client_fraction=0.75‘ 使节点选择机制真正参与到了训练调度中。这一差异决定了后续关于时延和通信收益的比较具有明确的工程含义。

5.3 评价指标与对照方案

为保证分析口径一致，本文沿用第2.4节中给出的评价指标定义，并结合当前工程原型的特点，将指标划分为检测性能、训练效率、通信效率与隐私代理四类。检测性能指标包括 Accuracy、Precision、Recall、F1 值和 False Positive Rate，用于衡量模型识别攻击样本的能力；训练效率指标包括总训练时间和平均轮次耗时，用于衡量联邦训练过程的整体时间开销；通信效率指标包括总原始通信量、总压缩后通信量和总体通信压缩率，用于评估参数压缩带来的直接收益；隐私代理指标则用于辅助说明“原始数据不出本地”与“上传更新量减少”对隐私暴露面的综合影响。

本章采用的对照方式遵循“单变量尽量少、工程意义尽量强”的原则。基线方案保留相同的模型结构和基本聚合逻辑，但让所有客户端都参与训练，并关闭压缩机制；改进方案在此基础上叠加节点选择、Top-K 压缩与 8 位量化，从而突出“调度 + 压缩”联

合优化的综合收益。这样设计的优点在于：比较双方共享相同的模型骨架和训练框架，便于将观察到的差异主要归因于客户端参与策略和参数上传方式的变化。

除现有两组结果外，实验设计文档还提出了集中式训练、FedProx 和不同压缩比例、多种客户端参与率等扩展实验思路，这些内容有助于后续正式论文形成更完整的实验矩阵。但在当前阶段，本文仅将其作为后续工作保留，不将其写作“已完成实验”。这种处理方式既满足学术写作中事实边界清晰的要求，也有利于后续在真实 VeReMi 或 Car-Hacking 数据集上继续补充对照结果。

5.4 实验结果与分析

根据 ‘outputs/reports/federated_training_report.md‘、‘outputs/baseline_fedavg/reports/federated_training 以及两组 ‘evaluation.json‘ 文件，可以得到改进方案与基线方案的主要对比结果，如表5-2所示。

表 5-2 改进方案与基线方案的关键结果对比

指标	改进方案	基线方案	对比结论
总训练时间（ms）	933.24	1063.74	下降约12.27%
平均轮次耗时（ms）	304.51	332.83	下降约8.51%
总通信量（bytes）	79848	404928	下降约80.28%
报告内通信压缩率	0.7371	0.0000	压缩效果明显
平均隐私代理分数	0.9211	0.7000	改进方案更高
Accuracy	0.60	0.60	保持一致
Precision	0.60	0.60	保持一致
Recall	1.00	1.00	保持一致
F1-score	0.75	0.75	保持一致
推理延迟（ms）	3.45	3.14	差异较小
False Positive Rate	1.00	1.00	均偏高
样本量	5	5	当前仅为演示验证

从通信侧结果看，改进方案的优势最为明显。由于系统在上传前仅保留 Top-K 关键增量，并进一步使用 8 位量化表达更新值，最终总通信量由基线方案的 404928 字节降至 79848 字节，下降幅度约为 80.28%。报告中给出的总体通信压缩率为 0.7371，也表明在单轮原始上传量作为参考的情况下，压缩机制能够显著减少模型更新传输开销。对于资源受限且链路波动明显的车联网场景而言，这一收益具有直接工程价值。

从训练侧结果看，改进方案的总训练时间和平均轮次耗时也均优于基线方案。其原因主要体现在两个方面：一方面，改进方案把客户端参与率控制在 0.75，每轮仅选择部分节点参与训练，减少了需要串行完成的本地训练任务数；另一方面，参数压缩降低了每轮更新的处理与统计开销。虽然平均本地训练耗时在报告中略高于基线方案，但系统整体训练时间依然下降，说明“减少参与客户端数 + 降低上传量”的综合收益超过了局部训练时间波动带来的影响。

从检测侧结果看，改进方案与基线方案在 Accuracy、Precision、Recall 和 F1-score 上保持一致，分别为 0.60、0.60、1.00 和 0.75。这意味着在当前演示实验条件下，引入节点选择与压缩机制并未导致这些核心检测指标下降。与此同时，推理延迟分别为 3.45 ms 和 3.14 ms，差异较小，说明改进方案主要影响训练过程和上传机制，而不会明显增加本地推理负担。因此，可以初步认为当前系统在演示条件下实现了“通信开销显著下降而检测效果基本保持”的目标。

不过，结果中也暴露出一个必须正视的问题，即两组方案的 False Positive Rate 均为 1.00。这与 ‘evaluation.json‘ 中样本量仅为 5 的事实相互对应，并结合混淆矩阵可知，当前验证集中两个负类样本均被误判为正类。换言之，虽然召回率达到 1.00，但系统在当前极小样本演示条件下对正常样本的区分能力并不理想。因此，本文更适合将现阶段实验结果解释为“工程链路和指标采集机制可用”，而不能据此得出系统在真实车联网环境中已经具备稳定高精度检测性能的强结论。

从隐私代理维度看，改进方案的平均分数为 0.9211，高于基线方案的 0.7000。考虑到当前代理指标由本地性基础分与压缩收益共同构成，这一结果说明在保持原始数据留在本地的同时，进一步压缩上传更新量可以在工程意义上减少暴露面。但如前所述，该分数仅用于辅助比较，并不等价于差分隐私、同态加密或安全多方计算等严格隐私证明。

5.5 局限性与有效性讨论

尽管当前实验已经能够支撑系统原型的功能验证和初步对照分析，但其局限性也非常明显。首先，当前实验使用的是仓库内置演示数据，最终评估样本量仅为 5，显然不足以代表真实车联网数据环境中的分布复杂性与攻击多样性。其次，现有结果主要集中在改进方案与标准 FedAvg 基线方案之间的单次对照，尚未对不同随机种子、不同参与率、不同 Top-K 比例和不同量化位宽开展系统性多轮重复实验，因此难以对结果稳定性给出统计意义上的判断。

其次，虽然实验设计文档提出了集中式训练、FedProx、多组压缩参数和多组节点

选择权重等扩展方案，但这些内容在当前仓库中还未形成完整的结果证据链。因此，正式论文阶段仍需在真实 VeReMi 或 Car-Hacking 数据集上重新组织实验矩阵，至少对关键方案重复运行多次，并汇报均值与标准差，以增强结论的可信度。同时，还应补充更大规模验证集或独立测试集，避免当前“验证集即测试依据”的局限。

从有效性角度看，当前实验仍然具有一定价值。其一，它证明了系统已经具备从数据预处理到联邦训练、从模型评估到证据报告生成的完整闭环；其二，它给出了压缩机制和节点筛选对通信与训练效率影响的直接量化结果；其三，它揭示了在小样本场景下误报率偏高这一问题，为后续正式实验和模型改进指明了方向。因此，本文将现阶段结果定位为“工程验证性结果”是合适且必要的。

5.6 本章小结

本章结合实验设计文档、改进方案报告、基线方案报告与评估结果，对系统的实验目标、参数设置、对照方式和结果表现进行了分析。结果表明，在当前演示实验条件下，引入动态节点选择、Top-K 稀疏化与定点量化后，系统总通信量下降约 80.28%，总训练时间下降约 12.27%，平均轮次耗时下降约 8.51%，而 Accuracy、Precision、Recall 与 F1-score 均未发生明显退化。同时，实验也暴露出样本量过小和误报率偏高等问题，说明现阶段结论更适合支持工程原型的有效性论证，而不宜直接外推到真实大规模车联网场景。下一章将在此基础上对全文工作进行总结，并给出后续扩展方向。

结论与展望

0.1 论文工作总结

本文围绕“基于联邦学习的车联网入侵检测系统设计与实现”这一题目，结合当前仓库中的项目代码、实验文档、运行产物与学校论文模板，完成了一套本科毕业论文初稿的正文组织，并对系统的设计思路、关键实现和实验结果进行了系统整理。与仅停留在算法描述层面的研究不同，本文特别强调工程闭环的完整性，即从原始数据预处理、联邦训练调度、模型评估到结果报告生成，都能够在现有代码中找到对应实现与输出证据。

在系统设计方面，本文以 ‘main.py‘ 为统一入口，将系统划分为预处理、训练和测试三种命令行模式，并通过 ‘config.py‘ 中的配置模型对数据路径、训练轮数、客户端数量、节点选择权重、Top-K 比例和量化位宽等关键参数进行统一管理。与此同时，项目还引入了随机种子设置、日志输出、配置快照与运行元数据保存机制，使得实验过程具备了较好的可复现性和可追踪性。这些设计不仅满足了车联网入侵检测原型的基本运行要求，也为论文写作与后续扩展打下了较清晰的系统基础。

在系统实现方面，本文详细分析了数据预处理模块、客户端切分与状态建模机制、联邦训练与聚合流程、参数压缩与通信统计方法，以及模型评估与证据报告落盘方式。当前系统采用轻量化 CNN-LSTM 作为检测模型，在联邦训练阶段实现了基于算力、电量和信道质量的动态节点选择机制，并结合轻量公平性惩罚缓解频繁选点问题；在模型上传阶段，则通过 Top-K 稀疏化与定点量化减少通信负担。整体上，项目已经形成了“预处理—联邦训练—本地评估—报告生成”的闭环原型，能够作为车联网联邦入侵检测系统的工程化基础实现。

在实验验证方面，本文基于仓库现有的改进方案与标准 FedAvg 基线方案结果进行了对照分析。实验结果表明，在当前演示数据和小规模验证场景下，改进方案能够在 Accuracy、Precision、Recall 和 F1-score 保持不变的同时，使总训练时间下降约 12.27%，平均轮次耗时下降约 8.51%，总通信量下降约 80.28%，平均隐私代理分数也高于基线方案。这说明动态节点选择、Top-K 压缩与量化机制在当前工程条件下具备较明显的综合收益，尤其在通信效率方面表现突出。

与此同时，本文也明确指出了当前系统和实验结果的边界。现有评估样本量较小，False Positive Rate 偏高，且尚未在真实规模的 VeReMi 或 Car-Hacking 数据集上完成多轮重复实验，因此现阶段结论更适合用于证明系统原型的可运行性与方法链路的可分析性，而不能直接外推为真实车联网环境中的最终性能结论。换言之，本文的主要贡献在

于完成了一套事实可核验的系统设计与实现整理，并给出了后续继续深化实验与完善文献的可靠起点。

0.2 工作展望

面向后续工作，本文认为至少可以从以下几个方面继续深化。第一，应尽快引入真实的 VeReMi、Car-Hacking 或同类车联网日志数据集，对现有系统进行多次重复实验，并报告均值、标准差及必要的显著性分析，以增强实验结论的可信度。第二，可围绕 FedProx、多种客户端参与率、多组 Top-K 比例与量化位宽，构造更完整的对照实验矩阵，从而更系统地分析不同改进策略之间的相互作用。第三，可进一步改进模型结构与特征工程，例如结合更细粒度的时序特征、异常模式特征或注意力机制，以降低当前小样本场景下误报率偏高的问题。

此外，若希望系统更接近真实车联网部署环境，还可以在未来工作中补充更真实的节点状态采集机制、在线数据流处理能力和更严格的隐私保护手段。例如，可将当前的隐私代理指标扩展为差分隐私或安全聚合等具备理论保障的方法；也可以将单机顺序模拟的联邦训练原型进一步扩展为多进程甚至多节点的真实联邦实验平台。对于论文写作本身，后续仍需结合开题报告、任务书和正式参考文献，对绪论中的研究现状、文献综述和题目背景进行进一步润色与补强，从而形成更加完整、更加符合正式答辩要求的毕业论文终稿。

参考文献

[1] 待补充作者. 车联网安全与攻击防护综述（待补充）[J]. 待补充期刊, 2025.

[2] 待补充作者. 车联网入侵检测技术综述（待补充）[J]. 待补充期刊, 2025.

[3] 待补充作者. 分布式入侵检测系统研究进展（待补充）[J]. 待补充期刊, 2025.

[4] 待补充作者. 隐私保护型网络入侵检测方法研究（待补充）[J]. 待补充期刊, 2025.

[5] KAIROUZ P, MCMAHAN H B, AVENT B, et al. Advances and open problems in federated learning[J]. Foundations and Trends in Machine Learning, 2021, 14(1–2): 1-210.

[6] MCMAHAN B, MOORE E, RAMAGE D, et al. Communication-efficient learning of deep networks from decentralized data[C]//Proceedings of the 20th International Conference on Artificial Intelligence and Statistics. 2017: 1273-1282.

[7] 开题报告与任务书待补材料. 本科毕业设计开题报告与任务书相关表述（待补充）[Z]. 2025.

[8] 待补充作者. 基于机器学习的入侵检测研究综述（待补充）[J]. 待补充期刊, 2025.

[9] 待补充作者. 基于深度学习的入侵检测研究综述（待补充）[J]. 待补充期刊, 2025.

[10] LI T, SAHU A K, TALWALKAR A, et al. Federated optimization in heterogeneous networks[C]//Proceedings of Machine Learning and Systems: Vol. 2. 2020: 429-450.

[11] 待补充作者. 联邦学习在车联网入侵检测中的应用研究（待补充）[J]. 待补充期刊, 2025.

[12] 待补充作者. 边缘智能与联邦安全分析研究（待补充）[J]. 待补充期刊, 2025.

致谢

时光荏苒，本科阶段的学习与研究工作即将画上阶段性句号。本文能够顺利完成，离不开老师、同学、家人以及项目资料提供者在学习、实践和写作过程中的支持与帮助。在此谨向所有给予关心和指导的人表示诚挚的感谢。

首先，衷心感谢指导教师（待补充姓名与职称）在课题选题、研究思路、系统实现和论文写作等方面给予的悉心指导。无论是在研究方向的把握、工程问题的分析，还是在学术表达的规范性方面，指导教师都给予了耐心的建议和启发，使我能够逐步完成从系统实现到论文整理的全过程。

其次，感谢在项目开发、实验验证和资料整理过程中给予帮助的老师和同学。大家在课程学习、代码调试、实验复现和问题讨论中的支持，为本文的完善提供了重要帮助。仓库中已有的论文模板、实验文档与运行产物，也为本研究的系统化整理提供了直接依据。

最后，感谢家人在学习与生活上的理解、鼓励与支持。正是这份长期的陪伴和信任，使我能够在面对困难和不确定性时保持耐心与信心，并最终完成本阶段的研究任务。

本致谢内容为通用草稿，后续可根据个人实际情况进一步补充具体姓名、经历与个性化表达，以形成正式提交版本。

附录 A 复现实验命令与结果文件说明

A.1 复现实验命令示例

本附录整理当前仓库中可直接复现的预处理、训练与测试命令，供正文第五章引用。以下命令均假定用户已在项目根目录完成依赖安装，并使用虚拟环境中的 Python 解释器执行。

A.1.1 数据预处理命令

预处理阶段用于生成训练集、验证集和标签映射：

```
./.venv/bin/python main.py --mode preprocess --config configs/default.toml
```

执行完成后，默认会在 ‘outputs/‘ 目录中生成 ‘train_processed.csv‘、‘val_processed.csv‘ 与 ‘label_mapping.json‘。

A.1.2 改进方案训练命令

为了与正文中的改进方案结果保持一致，可使用如下命令执行动态节点选择 + Top-K 压缩 + 8 位量化的联邦训练：

```
./.venv/bin/python main.py --mode train --config configs/default.toml \
  --output-dir outputs --rounds 3 --clients 4 --epochs 2 --batch-size 8 \
  --client-fraction 0.75 --topk-ratio 0.2 --quant-bits 8 \
  --selection-weight-compute 0.4 --selection-weight-battery 0.3 \
  --selection-weight-channel 0.3 --run-name evidence_demo
```

A.1.3 基线方案训练命令

标准 FedAvg 基线方案采用全客户端参与且关闭压缩上传，其结果建议单独输出到 ‘outputs/baseline_fedavg/‘ 目录：

```
./.venv/bin/python main.py --mode train --config configs/default.toml \
  --output-dir outputs/baseline_fedavg --rounds 3 --clients 4 --epochs 2 \
  --batch-size 8 --client-fraction 1.0 --topk-ratio 1.0 --quant-bits 32 \
  --selection-weight-compute 0.4 --selection-weight-battery 0.3 \
  --selection-weight-channel 0.3 --run-name baseline_fedavg
```

A.1.4 模型评估命令

训练完成后，可分别对改进方案和基线方案执行评估：

```
./.venv/bin/python main.py --mode test --config configs/default.toml \
  --output-dir outputs --rounds 3 --clients 4 --epochs 2 --batch-size 8 \
  --run-name evidence_demo
```

```
./.venv/bin/python main.py --mode test --config configs/default.toml \
  --output-dir outputs/baseline_fedavg --rounds 3 --clients 4 --epochs 2 \
  --batch-size 8 --run-name baseline_fedavg
```

A.2 关键输出文件与证据路径

为了支撑正文中的实验分析，当前项目将关键结果文件统一保存在 ‘outputs/‘ 及其子目录中。主要文件说明如表A-1所示。

表 A-1 附录中的关键输出文件说明

文件路径	说明
‘outputs/train_processed.csv‘	预处理后的训练集文件。
‘outputs/val_processed.csv‘	预处理后的验证集文件，也是当前测试模式使用的数据来源。
‘outputs/label_mapping.json‘	标签编码映射文件。
‘outputs/global_model.pt‘	改进方案生成的全局模型检查点。
‘outputs/training_history.json‘	改进方案的逐轮训练历史。
‘outputs/reports/evaluation.json‘	改进方案测试评估结果。
‘outputs/reports/federated_training_report.json‘	改进方案的结构化联邦训练报告。
‘outputs/reports/federated_training_report.md‘	改进方案的 Markdown 证据报告。
‘outputs/baseline_fedavg/reports/evaluation.json‘	基线方案测试评估结果。
‘outputs/baseline_fedavg/reports/federated_training_report.md‘	基线方案的 Markdown 证据报告。

其中，正文第5.4节主要使用了以下文件作为分析证据：‘outputs/reports/federated_training_report.md‘、‘outputs/baseline_fedavg/reports/federated_training_report.md‘、‘outputs/reports/evaluation.json‘ 和 ‘outputs/baseline_fedavg/reports/evaluation.json‘。

A.3 关键配置与参数补充说明

默认配置文件 ‘configs/default.toml‘ 为系统提供了可运行的基础参数，其核心内容包括：数据集名称 ‘VeReMi & Car-Hacking‘、标签列 ‘label‘、验证集比例 ‘0.2‘、客户端数

量 ‘4‘、默认联邦轮数 ‘5‘、默认 Top-K 比例 ‘0.1‘、量化位宽 ‘8‘ 以及节点选择权重 ‘0.4 / 0.3 / 0.3‘。正文实验部分为了对比方便，将轮数调整为 3，并对改进方案与基线方案分别使用不同的客户端参与率、Top-K 比例和量化位宽。

附录中列出的命令与参数主要用于复现实验链路和验证已有结果文件，不代表正式论文最终实验配置已经完备。后续若引入真实数据集、多分类攻击标签或更多对照组，需要同步更新命令示例、结果目录和参数表，以保证正文、附录和产物路径始终一致。